

views::join_with

Document #: P2441R1
Date: 2021-11-13
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Design](#)
 - [3.1 Naming](#)
 - [3.2 Category](#)
 - [3.3 Value and Reference Type](#)
 - [3.4 Various Properties](#)
 - [3.4.1 Conditionally Common](#)
 - [3.4.2 Borrowed](#)
 - [3.4.3 Sized](#)
 - [3.4.4 const-iterable](#)
 - [3.5 Implementation Experience](#)
- [4 Wording](#)
 - [4.1 Addition to <ranges>](#)
 - [4.2 join_with](#)
 - [24.7.? Join with view \[range.join_with\]](#)
 - [4.3 Feature-test macro](#)
- [5 References](#)

§ 1 Revision History

Since [\[P2441R0\]](#), added a feature-test macro.

§ 2 Introduction

C++20 Ranges introduced `views::join`: a range adaptor for turning a range of range of T into a range of T . This adaptor, however, did not take any delimiter range. So you could not, for instance, join together a range of strings with a space to then convert that to a longer string. This paper remedies that by producing a `views::join_with`, as described in [\[P2214R1\]](#), section 3.8.

The behavior of `views::join_with` is an inverse of `views::split`. That is, given a range `r` and a pattern `p`, `r | views::split(p) | views::join_with(p)` should yield a range consisting of the same elements as `r`.

§ 3 Design

There are several aspects to `join_with` that are more complicated than `join`, that are worth going over. For convenience, let `Rng` denote the range we're joining (i.e. the range of ranges), let `Pattern` denote the delimiter pattern, and let `Inner` denote the inner range (i.e. `range_reference_t<Rng>`).

§ 3.1 Naming

Why `views::join_with` rather than supporting an extra argument to `views::join`? The issue here is ultimately ambiguity. In this potential design where we simply overload `join`, `views::join(x)` could mean either (a) produce a range that joins the ranges of `x`, or (b) it is a partial call that produces a range adaptor closure object to where `x` is a delimiter for some other range to be provided in the future.

This ambiguity is... rare. `x` needs to be a range of ranges (i.e. `[[T]]`) in order to be joinable, which means that in order for it to be a delimiter, the range it's doing needs to be a range of range of ranges (i.e. `[[[T]]]`). These don't come up very often. `range-v3` simply treats `join(x)` if `x` is a joinable range (i.e. a range whose reference type is also a range) a request to produce a `join_view`. That's one option.

However, eventually somebody is going to want to join a `[[[T]]]` on a `[[T]]` and it will end up just not working.

Regardless, joining with a delimiter will produce a different view than joining without a delimiter (it's `r | views::join` does not produce a `join_with_view<all_t<R>, empty_view<T>>`, that would be needlessly inefficient... it just produces a `join_with_view<all_t<R>>`), so we don't gain anything on either the implementation front or the specification front by overloading `join`. The only difference of introducing a new name would be that users would have to write `r | views::join_with(' ')` instead of `r | views::join(' ')`. That doesn't seem like an unreasonable burden on users, at the cost of avoiding any future ambiguity.

§ 3.2 Category

Like `views::join`, and following [\[P2328R1\]](#), `views::join_with` is input-only if the primary range (the one we're joining) is a range of prvalue non-view ranges.

Otherwise, if `Rng`, `Inner`, and `Pattern` are all bidirectional and `Inner` and `Pattern` are common, then bidirectional. Otherwise, if `Rng`, `Inner`, and `Pattern` are all forward. Otherwise, input.

§ 3.3 Value and Reference Type

For `views::join`, these are obvious: `Inner`'s value and reference, respectively. But now, we have two ranges that we're joining together: `Inner` and `Pattern`, and those two may have different value and reference types. `range-v3`'s implementation allows this, using the following formation:

```
using value_type = common_type_t<range_value_t<Inner>, range_value_t<Pattern>>;
using reference = common_reference_t<range_reference_t<Inner>,
    range_reference_t<Pattern>>;
using rvalue_reference = common_reference_t<range_rvalue_reference_t<Inner>,
    range_rvalue_reference_t<Pattern>>;
```

Those types all existing is an added constraint on constructing a `join_with_view`.

I'm not sure I see a need to deviate from the implementation here.

§ 3.4 Various Properties

§ 3.4.1 Conditionally Common

Like `views::join`, `views::join_with` can be common when `Rng` is a range of glvalue ranges, `Rng` and `Inner` are both forward and common. Note that we don't need `Pattern` to be common.

§ 3.4.2 Borrowed

Never.

§ 3.4.3 Sized

Never.

§ 3.4.4 `const-iterable`

If `Rng` and `Pattern` are, and `Inner` is glvalue range. This is the same requirement as `join_view` has, because if `Inner` is not a reference that means we have to store it, which requires state, which implies mutating that state during iteration.

§ 3.5 Implementation Experience

Up until recently, the `join_with_view` in `range-v3` was input-only, never common, and never const-iterable. I have implemented conditionally-bidirectional support in `range-v3` and also implemented [this design](#) from scratch.

§ 4 Wording

§ 4.1 Addition to `<ranges>`

Add the following to 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis:

```
// [...]
namespace std::ranges {
    // [...]

    // [range.join.with], join with view
    template<class R, class P>
```

```

    concept compatible-joinable-ranges = see below; // exposition only

template<input_range V, forward_range Pattern>
    requires view<V>
        && input_range<range_reference_t<V>>
        && view<Pattern>
        && compatible-joinable-ranges<range_reference_t<V>, Pattern>
class join_with_view;

namespace views {
    inline constexpr unspecified join_with = unspecified;
}
}

```

§ 4.2 join_with

Add the following subclause to 24.7 [\[range.adaptors\]](#).

§ 24.7.? Join with view [\[range.join.with\]](#)

§ 24.7.?.1 Overview [\[range.join.with.overview\]](#)

- 1 join_with_view takes a view and a delimiter, and flattens the view, inserting every element of the delimiter in between elements of the view. The delimiter can be a single element or a view of elements.
- 2 The name views::join_with denotes a range adaptor object ([\[range.adaptor.object\]](#)). Given subexpressions E and F, the expression views::join_with(E, F) is expression-equivalent to join_with_view(E, F).
- 3 *[Example:*

```

vector<string> vs = {"the", "quick", "brown", "fox"};
for (char c : vs | join_with('-')) {
    cout << c;
}
// the above prints: the-quick-brown-fox

```

-end example]

§ 24.7.?.2 Class template join_with_view [\[range.join.with.overview\]](#)

```

namespace std::ranges {
    template<class R, class P>
        concept compatible-joinable-ranges = // exposition only
            common_with<range_value_t<R>, range_value_t<P>> &&
            common_reference_with<range_reference_t<R>, range_reference_t<P>> &&
            common_reference_with<range_rvalue_reference_t<R>,
            range_rvalue_reference_t<P>>;

    template <class R>

```

```

concept bidi-common = bidirectional_range<R> && common_range<R>; //
    exposition only

template<input_range V, forward_range Pattern>
    requires view<V>
        && input_range<range_reference_t<V>>
        && view<Pattern>
        && compatible-joinable-ranges<range_reference_t<V>, Pattern>
class join_with_view : public view_interface<join_with_view<V, Pattern>> {
    using InnerRng = range_reference_t<V>;

    V base_ = V(); // exposition only
    non-propagating-cache<remove_cv_t<InnerRng>> inner_; // exposition only, present only
    // when
    // !is_reference_v<InnerRng>
    Pattern pattern_ = Pattern(); // exposition only

    template<bool Const> struct iterator; // exposition only
    template<bool Const> struct sentinel; // exposition only

public:
    join_with_view() requires default_initializable<V> &&
        default_initializable<Pattern> = default;
    constexpr join_with_view(V base, Pattern pattern);

    template<input_range R>
        requires constructible_from<V, views::all_t<R>> &&
            constructible_from<Pattern,
                single_view<range_value_t<InnerRng>>>
    constexpr join_with_view(R&& r, range_value_t<InnerRng> e);

    constexpr V base() const& requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() {
        constexpr bool use_const = simple-view<V> && is_reference_v<InnerRng>
            && simple-view<Pattern>;
        return iterator<use_const>{*this, ranges::begin(base_)};
    }
    constexpr auto begin() const requires input_range<const V> &&
        forward_range<const Pattern> &&
        is_reference_v<InnerRng> {
        return iterator<true>{*this, ranges::begin(base_)};
    }

    constexpr auto end() {
        if constexpr (forward_range<V> &&
            is_reference_v<InnerRng> && forward_range<InnerRng> &&
            common_range<V> && common_range<InnerRng>) {

```

```

        return iterator<simple-view<V> && simple-view<Pattern>>{*this,
            ranges::end(base_)};
    } else {
        return sentinel<simple-view<V> && simple-view<Pattern>>{*this};
    }
}

constexpr auto end() const requires input_range<const V> &&
                                   forward_range<const Pattern> &&

    is_reference_v<range_reference_t<const V>> {
if constexpr (forward_range<const V> &&
               is_reference_v<range_reference_t<const V>> &&
               forward_range<range_reference_t<const V>> &&
               common_range<V> && common_range<range_reference_t<const
V>>>) {
    return iterator<true>{*this, ranges::end(base_)};
} else {
    return sentinel<true>{*this};
}
}
};

template<class R, class P>
join_with_view(R&&, P&&) -> join_with_view<views::all_t<R>,
    views::all_t<P>>;

template<input_range R>
join_with_view(R&&, range_value_t<range_reference_t<R>>)
    -> join_with_view<views::all_t<R>,
        single_view<range_value_t<range_reference_t<R>>>>;
}

```

```
constexpr join_with_view(V base, Pattern pattern);
```

- 1 *Effects:* Initializes *base_* with `std::move(base)` and *pattern_* with `std::move(pattern)`.

```

template<input_range R>
requires constructible_from<V, views::all_t<R>> &&
         constructible_from<Pattern, single_view<range_value_t<InnerRng>>>
constexpr join_with_view(R&& r, range_value_t<InnerRng> e);

```

- 2 *Effects:* Initializes *base_* with `views::all(std::forward<R>(r))` and *pattern_* with `views::single(std::move(e))`.

§ 24.7.?.3 Class template `join_with_view::iterator` [range.join.with.iterator]

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
    requires view<V>
             && input_range<range_reference_t<V>>
             && view<Pattern>
             && compatible_joinable_ranges<range_reference_t<V>, Pattern>
    template<bool Const>
    struct join_with_view<V, Pattern>::iterator {

```

```

using Parent = maybe-const<Const, join_with_view>; //
    exposition only
using Base = maybe-const<Const, V>; //
    exposition only
using InnerBase = range_reference_t<Base>; //
    exposition only
using PatternBase = maybe-const<Const, Pattern>; //
    exposition only

using OuterIter = iterator_t<Base>; //
    exposition only
using InnerIter = iterator_t<InnerBase>; //
    exposition only
using PatternIter = iterator_t<PatternBase>; //
    exposition only

static constexpr bool ref-is-glvalue = is_reference_v<InnerBase>; //
    exposition only

Parent* parent_ = nullptr; //
    exposition only
OuterIter outer_it_ = OuterIter(); //
    exposition only
variant<PatternIter, InnerIter> inner_it_; //
    exposition only

constexpr auto&& update-inner(const OuterIter&); //
    exposition only
constexpr decltype(auto) get-inner(const OuterIter&); //
    exposition only
constexpr void satisfy(); //
    exposition only
public:
    using iterator_concept = see below;
    using iterator_category = see below; // not
        always present
    using value_type = see below;
    using difference_type = see below;

    iterator() requires default_initializable<OuterIter> = default;
    constexpr iterator(Parent& parent, iterator_t<Base> outer);
    constexpr iterator(iterator<!Const> i)
        requires Const &&
            convertible_to<iterator_t<V>, OuterIter> &&
            convertible_to<iterator_t<InnerRng>, InnerIter> &&
            convertible_to<iterator_t<Pattern>, PatternIter>;

    constexpr decltype(auto) operator*() const;

    constexpr iterator& operator++();
    constexpr void operator++(int);
    constexpr iterator operator++(int)
        requires ref-is-glvalue && forward_iterator<OuterIter> &&
            forward_iterator<InnerIter>;

    constexpr iterator& operator--()
        requires ref-is-glvalue && bidirectional_range<Base> &&

```

```

        bidi-common<InnerBase> && bidi-common<PatternBase>;
constexpr iterator operator--(int)
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidi-common<InnerBase> && bidi-common<PatternBase>;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && equality_comparable<OuterIter> &&
        equality_comparable<InnerIter>;

friend constexpr decltype(auto) iter_move(const iterator& x)
{
    using rvalue_reference = common_reference_t<
        iter_rvalue_reference_t<InnerIter>,
        iter_rvalue_reference_t<PatternIter>>;
    return std::visit<rvalue_reference>(ranges::iter_move, x.inner_it_);
}

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    requires indirectly_swappable<InnerIter, PatternIter>
{
    std::visit(ranges::iter_swap, x.inner_it_, y.inner_it_);
}
};
}

```

1 *iterator*::*iterator_concept* is defined as follows:

- (1.1) — If *ref-is-glvalue* is *true*, *Base* models *bidirectional_range*, and *InnerBase* and *PatternBase* each model *bidi-common*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.2) — Otherwise, if *ref-is-glvalue* is *true* and *Base* and *InnerBase* each model *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

2 The member *typedef-name* *iterator_category* is defined if and only if *ref-is-glvalue* is *true*, and *Base* and *InnerBase* each model *forward_range*. In that case, *iterator*::*iterator_category* is defined as follows:

- (2.1) — Let OUTER_C denote *iterator_traits*<OuterIter>::*iterator_category*. Let INNER_C denote *iterator_traits*<InnerIter>::*iterator_category*, and let PATTERN_C denote *iterator_traits*<PatternIter>::*iterator_category*.
- (2.2) — If *is_lvalue_reference_v*<*common_reference_t*<*iter_reference_t*<InnerIter>, *iter_reference_t*<PatternIter>>> is *false*, *iterator_category* denotes *input_iterator_tag*.
- (2.3) — Otherwise, if OUTER_C, INNER_C, and PATTERN_C each model *derived_from*<*bidirectional_iterator_category*> and *InnerBase* and *PatternBase* each model *common_range*, *iterator_category* denotes *bidirectional_iterator_tag*.

- (2.4) — Otherwise, if OUTER, INNER, and PATTERN each model `derived_from<forward_iterator_tag>`, `iterator_category` denotes `forward_iterator_tag`.
- (2.5) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

3 `iterator::value_type` denotes the type:

```
common_type_t<
    iter_value_t<InnerIter>,
    iter_value_t<PatternIter>>
```

4 `iterator::difference_type` denotes the type:

```
common_type_t<
    iter_difference_t<OuterIter>,
    iter_difference_t<InnerIter>,
    iter_difference_t<PatternIter>>
```

```
constexpr auto&& update-inner(const OuterIter& x); // exposition only
```

5 *Effects:* Equivalent to:

```
if constexpr (ref-is-glvalue) {
    return *x;
} else {
    return parent_ -> inner_.emplace-deref(x);
}
```

```
constexpr decltype(auto) get-inner(const OuterIter& x); // exposition only
```

6 *Effects:* Equivalent to:

```
if constexpr (ref-is-glvalue) {
    return *x;
} else {
    return *parent_ -> inner_;
}
```

7 `join_with_view` iterators use the *satisfy* function to skip over empty inner ranges.

```
constexpr void satisfy(); // exposition only
```

8 *Effects:* Equivalent to:

```
while (true) {
    if (inner_it_.index() == 0) {
        if (std::get<0>(inner_it_) != ranges::end(parent_ -
            >pattern_)) {
            break;
        }

        auto&& inner = update-inner(outer_it_);
        inner_it_.emplace<1>(ranges::begin(inner));
    } else {
```

```

        auto&& inner = get-inner();
        if (std::get<1>(inner_it_) != ranges::end(inner)) {
            break;
        }

        if (++outer_it_ == ranges::end(parent_>base_)) {
            if constexpr (ref-is-glvalue) {
                inner_it_ = {};
            }
            break;
        }

        inner_it_.emplace<0>(ranges::begin(parent_>pattern_));
    }
}

```

```
constexpr iterator(Parent& parent, iterator_t<Base> outer);
```

- 9 *Effects:* Initializes *parent_* with `std::addressof(parent)` and *outer_it_* with `std::move(outer)`. Then, equivalent to:

```

if (outer_it_ != ranges::end(parent_>base_)) {
    auto&& inner = update-inner(outer_it_);
    inner_it_.emplace<1>(ranges::begin(inner));
    satisfy();
}

```

```

constexpr iterator(iterator<!Const> i)
    requires Const &&
        convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter> &&
        convertible_to<iterator_t<Pattern>, PatternIter>;

```

- 10 *Effects:* Initializes *outer_it_* with `std::move(i.outer_it_)` and *parent_* with `i.parent_`. Then, equivalent to:

```

if (i.inner_it_.index() == 0) {
    inner_it_.emplace<0>(std::get<0>(std::move(i.inner_it_)));
} else {
    inner_it_.emplace<1>(std::get<1>(std::move(i.inner_it_)));
}

```

```
constexpr decltype(auto) operator*() const;
```

- 11 *Effects:* Equivalent to:

```

using reference = common_reference_t<
    iter_reference_t<InnerIter>,
    iter_reference_t<PatternIter>>;
return std::visit([](auto& it) -> reference { return *it; },
    inner_it_);

```

```
constexpr iterator& operator++();
```

- 12 *Effects:* Equivalent to:

```
std::visit([](auto& it){ ++it; }, inner_it_);
satisfy();
return *this;
```

```
constexpr void operator++(int);
```

13 *Effects:* Equivalent to `++*this`;

```
constexpr iterator operator++(int)
    requires ref-is-glvalue && forward_iterator<OuterIter> &&
        forward_iterator<InnerIter>;
```

14 *Effects:* Equivalent to:

```
iterator tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidi-common<InnerBase> && bidi-common<PatternBase>;
```

15 *Effects:* Equivalent to:

```
if (outer_it_ == ranges::end(parent_>base_)) {
    inner_it_.emplace<1>(ranges::end(*--outer_it_));
}

while (true) {
    if (inner_it_.index() == 0) {
        auto& it = std::get<0>(inner_it_);
        if (it == ranges::begin(parent_>pattern_)) {
            inner_it_.emplace<1>(ranges::end(*--outer_it_));
        } else {
            break;
        }
    } else {
        auto& it = std::get<1>(inner_it_);
        if (it == ranges::begin(*outer_it_)) {
            inner_it_.emplace<0>(ranges::end(parent_>pattern_));
        } else {
            break;
        }
    }
}

std::visit([](auto& it){ --it; }, inner_it_);
return *this;
```

```
constexpr iterator operator--(int)
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidi-common<InnerBase> && bidi-common<PatternBase>;
```

16 *Effects:* Equivalent to:

```

iterator tmp = *this;
--*this;
return tmp;

```

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && equality_comparable<OuterIter> &&
        equality_comparable<InnerIter>;

```

- 17 *Effects:* Equivalent to `return x.outer_it_ == y.outer_it_ && x.inner_it_ == y.inner_it_;`

§ 24.7.?.3 Class template `join_with_view::sentinel` [range.join.with.sentinel]

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V>
            && input_range<range_reference_t<V>>
            && view<Pattern>
            && compatible_joinable_ranges<range_reference_t<V>, Pattern>
        template<bool Const>
        struct join_with_view<V, Pattern>::sentinel {
            using Parent = maybe_const<Const, join_with_view>; // exposition only
            using Base = maybe_const<Const, V>; // exposition only
            sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only

            sentinel() = default;
            constexpr explicit sentinel(Parent& parent);
            constexpr sentinel(sentinel<!Const> s)
                requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

            template <bool OtherConst>
                requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-
                    const<OtherConst, V>>>
            friend constexpr bool operator==(const iterator<OtherConst>& x, const
                sentinel& y);
        };
}

```

```
constexpr explicit sentinel(Parent& parent);
```

- 1 *Effects:* Initializes `end_` with `ranges::end(parent.base_)`.

```

constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

- 2 *Effects:* Initializes `end_` with `std::move(s.end_)`.

```

template <bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-
        const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const
    sentinel& y);

```

- 3 *Effects:* Equivalent to `return x.outer_it_ == y.end_;`

§ 4.3 Feature-test macro

Add the following macro definition to 17.3.2 [\[version.syn\]](#), with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_join_with 20XXXXL // also in <ranges>
```

§ 5 References

[P2214R1] Barry Revzin, Conor Hoekstra, and Tim Song. 2021. A Plan for C++23 Ranges.

<https://wg21.link/p2214r1>

[P2328R1] Tim Song. 2021-05-08. `join_view` should join all views of ranges.

<https://wg21.link/p2328r1>

[P2441R0] Barry Revzin. 2021-09-15. `views::join_with`.

<https://wg21.link/p2441r0>